

DATA_MODEL

Helios “ Data Model

DATA_MODEL.md · Companion to HELIOS_PROJECT_BIBLE.md · “11, DBT_GUIDE.md , and models/semantic/semantic_layer.yaml ·
Version: v1.0 · Date: 2026-06-03

Purpose. This is the canonical, production-grade data model for Helios “ the single reference for *what tables exist, at what grain, with which keys, who owns them, and why each one exists*. Helios transforms raw GA4 e-commerce events into a Kimball-style star schema (conformed dimensions + fact tables) through a layered dbt pipeline (raw “ staging “ intermediate “ marts “ semantic). The marts defined here are the substrate the semantic layer exposes as metrics and the agents diagnose against “ so the grain discipline, primary keys, and foreign keys below are load-bearing for the entire product.

How to use this document

- Read “3 first for the whole picture (layered model “ ER diagram “ master catalog), then the per-model deep dives (“4 Event, “5 Session, “6 User, “7 Product, “8 Order).
- Every table is documented with **grain “ primary key “ foreign keys “ owner/steward “ why it exists**.
- This model is consistent with DBT_GUIDE.md (the build-time spec) and feeds the 5 grains the semantic layer queries: fct_funnel , fct_sessions , fct_orders , fct_order_items , fct_cohorts .

Conventions & keys (cheat-sheet)

Item	Rule
Session key	<code>session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))</code> ; a session = (user_pseudo_id, ga_session_id)
User key	user_pseudo_id “ cookie-grain (user_id is ~always NULL “ no cross-device stitching)
Order key	transaction_id (deduped)
Funnel flags	reached_* are max-downstream monotonic “ sessions “ reached_view_item “ “ reached_purchase
Channels	10 GA4 default groups via one channel_group_case() macro; traffic_source is user first-touch (gotcha)
Money	*_in_usd only; rates as SUM(num)/SUM(den) after grouping
Grains of record	session (fct_funnel / fct_sessions), transaction (fct_orders); marts are wide (denormalized dims)

Table of Contents

1. Overview & Layered Model
2. Entity Relationship Diagram
3. Master Table Catalog
4. Event Model
5. Session Model
6. User Model
7. Product Model
8. Order Model

1. Overview & Layered Model

Helios transforms raw GA4 e-commerce telemetry into a governed, Kimball-style star schema that an autonomous multi-agent system can query *without ever authoring SQL by hand*. The data model is the foundation under everything else: the semantic layer (semantic_layer.yaml , 47 metrics)

1.1 The layered DAG

[illegible]

- **Raw source** (`src_ga4.events`) is the immutable source of truth owned by Google's GA4 export. Its rows are *events* with deeply nested `event_params`, `items[]`, and `ecommerce` structures. We read it exclusively through `source()` so that exactly one model touches the export schema; nothing downstream depends on the raw layout.
- **Staging** is the 1:1 typed/renamed access layer. It surfaces the scalar columns analysts actually need (`session_key`, `ga_session_id`, `page_location`, session-scoped source/medium, `device.*`, `geo.*`, `transaction_id`, `purchase_revenue_in_usd`) and flattens the nested `event_params` array exactly once so no downstream model ever re- `UNNEST` s. Staging isolates the entire pipeline from raw schema drift; materialized as cheap, always-fresh views.
- **Intermediate** holds the business logic that the raw export omits. GA4 ships no session row, so `int_ga4_sessionized` reconstructs one by grouping events on (`user_pseudo_id`, `ga_session_id`) and `int_ga4_funnel_steps` computes the six monotonic `reached_*` funnel flags. These two are the *keystones*: get them wrong and every downstream number is silently wrong. They are ephemeral/view â€œ never exposed to BI.
- **Marts** are the conformed star schema: wide, denormalized fact tables surrounded by conformed dimensions, materialized as tables (incremental on the large facts) so the autonomous run hits a fixed, predictable BigQuery byte budget.
- **Semantic** is a thin governed view over the marts. It is the *only* path to SQL â€œ `semantic-mcp.build_query` maps a metric name plus dimensions to mart columns, so an agent that asks for `revenue by channel_group` never sees a `JOIN` or a `SUM`.

1. **The session** `â€œ` one analytical visit, keyed by `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`. The session is split across two co-grained facts: `fact_sessions` carries engagement and the wide session descriptive dimensions, while `fact_funnel` carries the monotonic `reached_*` flags plus deduped `session_revenue`. `fact_funnel` is the **primary** session grain the semantic layer queries `â€œ` funnel, conversion, and revenue-per-session metrics all resolve there. The two join 1:1 on `session_key`: the funnel *extends* the session.

2. **The transaction** â€” one completed order, keyed by `order_key = transaction_id`. `fct_orders` is the deduped order header (gross/net revenue, refund, shipping, tax, quantity) and the entity of record for all order-grain financials. Its line items explode into `fct_order_items`.

A third, derived analytical entity â€” the **user** (`user_pseudo_id`) â€” is conformed in `dim_users` but is deliberately *cookie-grain*: `user_id` is almost always NULL in this dataset, so there is no cross-device stitching. One human on a phone and a laptop is two users. This is carried as an honesty caveat on every user-grain metric (ARPU, new/returning users) â€” they are per-cookie approximations, never true person counts.

1.3 Conformed dimensions

Four dimensions conform across the facts so the Decompose agent can pivot any metric across any slice without re-deriving the slice:

- **dim_date** â€” the date spine (2020-11-01..2021-01-31) with week/month/quarter/day-of-week/is_weekend for time-grain rollups.
- **dim_users** â€” the user entity-of-record (cookie-grain) with first-touch attribution and lifetime rollups.
- **dim_channels** â€” the 10 GA4 default channel groups produced by the single `channel_group_case()` macro, plus `is_paid` / `is_organic` / `channel_order`.
- **dim_items** â€” the product catalog (item name, brand, category 1..5, current price).

Because these dimensions are *conformed* (one shared definition, joined by surrogate key), a finding sliced by `device_category` in the funnel and a finding sliced by `device_category` in revenue use the same dimension values â€” the precondition for honest mix-vs-rate decomposition.

1.4 Mart â†’ semantic grain mapping

The semantic layer resolves every one of its 47 metrics against exactly **five base grains**. The mapping is explicit and 1:1:

Semantic grain	Backing mart	Entity	Metrics resolved here (examples)
<code>fct_funnel</code>	<code>fct_funnel</code>	session	<code>sessions</code> , <code>users</code> , <code>new_users</code> , <code>returning_users</code> , <code>session_conversion_rate</code> , <code>view_to_cart_rate</code> , <code>revenue</code> , <code>revenue_per_session</code> , <code>revenue_per_user</code>
<code>fct_sessions</code>	<code>fct_sessions</code>	session	<code>engaged_sessions</code> , <code>engagement_rate</code>
<code>fct_orders</code>	<code>fct_orders</code>	order	<code>orders</code> , <code>gross_revenue</code> , <code>net_revenue</code> , <code>aov</code> , <code>items_per_transaction</code>
<code>fct_order_items</code>	<code>fct_order_items</code>	order_item	product-level revenue, attach rate, units
<code>fct_cohorts</code>	<code>fct_cohorts</code>	cohort	<code>day_1_retention</code> , <code>day_7_retention</code> , <code>day_30_retention</code>

The remaining marts â€” `dim_*`, `fct_daily_funnel`, `fct_funnel_by_dim` â€” are conformed dimensions or pre-aggregated rollups the agents read *indirectly*. `fct_daily_funnel` and `fct_funnel_by_dim` are additive aggregations of `fct_funnel` that feed the Monitor (anomaly detection), Decompose (mix-shift), and the eval injector; they store additive counts only, and all *rates* are computed in the semantic layer from those counts so they re-aggregate correctly across any slice.

1.5 The Kimball / wide-fact philosophy

Helios follows dimensional modelling deliberately. Each fact table declares exactly one grain; conformed dimensions are shared by surrogate key; and the facts are **wide** â€” descriptive dimensions (`device_category`, `country`, `channel_group`, `is_new_user`, `landing_page`) are denormalized directly onto `fct_funnel` / `fct_sessions` / `fct_orders`. The semantic layer therefore slices a metric by simply `GROUP BY`-ing a column on the same fact, with no join at query time. That is what lets a Sonnet-class agent request "revenue by `channel_group` and `device_category` for last week" and get governed, reconciled SQL the agent never wrote: the breadth lives in the mart, not in ad-hoc query construction. Wide facts trade a little storage and ETL discipline for zero analyst-authored joins, which is precisely the property the grounding principle ("the LLM never authors raw SQL") requires.

2. Entity Relationship Diagram

2.1 Mermaid ER diagram

```
erDiagram
    dim_date {
        DATE date_key PK
        INT64 week
        INT64 month
        INT64 quarter
        STRING day_of_week
        BOOL is_weekend
    }
```

```

}
dim_users {
  STRING user_key PK "= user_pseudo_id"
  STRING first_channel_key FK
  TIMESTAMP first_touch_ts
  INT64 total_sessions
  FLOAT64 total_revenue
  BOOL is_purchaser
  BOOL is_new_user
}
dim_channels {
  STRING channel_key PK
  STRING channel_group
  BOOL is_paid
  BOOL is_organic
  INT64 channel_order
}
dim_items {
  STRING item_key PK "= item_id"
  STRING item_name
  STRING item_brand
  STRING item_category
  FLOAT64 current_price_in_usd
}
fct_sessions {
  STRING session_key PK
  STRING user_pseudo_id FK
  DATE date_key FK
  STRING channel_key FK
  BOOL engaged_session
  STRING device_category
  STRING country
}
fct_funnel {
  STRING session_key PK "FK → fct_sessions"
  STRING user_pseudo_id FK
  DATE date_key FK
  STRING channel_key FK
  BOOL reached_view_item
  BOOL reached_add_to_cart
  BOOL reached_purchase
  FLOAT64 session_revenue
}
fct_orders {
  STRING order_key PK "= transaction_id"
  STRING session_key FK
  STRING user_pseudo_id FK
  DATE date_key FK
  STRING channel_key FK
  FLOAT64 gross_revenue
  FLOAT64 net_revenue
  INT64 total_item_quantity
}
fct_order_items {
  STRING order_item_key PK
  STRING order_key FK
  STRING item_key FK
  INT64 quantity
  FLOAT64 item_revenue_in_usd
}
fct_daily_funnel {
  STRING daily_funnel_key PK
  DATE date_key FK
  STRING channel_key FK
  INT64 sessions
  INT64 purchasing_sessions
  FLOAT64 revenue
}
fct_funnel_by_dim {
  STRING funnel_by_dim_key PK
  DATE date_key FK
  STRING dimension
  STRING dimension_value
}
fct_cohorts {
  STRING cohort_key PK
  DATE cohort_week
  INT64 age_days
  INT64 cohort_size
  INT64 retained_users
}

```


- **fct_sessions 1 â€œâ€œ 1 fct_funnel (session_key).** The funnel *extends* the session: exactly one funnel row per session row, same session_key . fct_funnel is the primary semantic grain; fct_sessions adds engagement. Splitting them keeps each fact single-purpose while preserving a clean 1:1 join.
- **fct_sessions 1 â€œâ€œ 0..N fct_orders (session_key).** A session yields zero or more orders â€œ almost always 0 (most sessions don't buy) or 1. This is the bridge that lets revenue-per-session join order revenue back to the session universe.
- **fct_orders 1 â€œâ€œ * fct_order_items (order_key).** One deduped order header explodes into one row per item line. Order-level totals live on the header; line-level revenue lives on the items.
- **Aggregation rollups (NOT FKs).** fct_funnel is aggregated into fct_daily_funnel (additive daily counts + revenue, the feed for Monitor/Decompose and the eval injector) and fct_funnel_by_dim (funnel counts by one canonical dimension, the mix-vs-rate decomposition input). These are GROUP BY derivatives, carrying their own surrogate PKs and a date_key FK, but no row-level foreign key back to fct_funnel .

3. Master Table Catalog

Every table in the Helios pipeline, from raw source through the growth marts. Use these exact names, PKs, FKs, grains, and owners everywhere; the ER diagram above and every per-model section that follows must agree with this catalog.

Table	Layer	Grain	Primary Key	Foreign Keys	Owner / Steward	Why it exists
src_ga4.events	raw	one row per EVENT	none enforced (natural: user_pseudo_id , event_timestamp , event_name , event_bundle_sequence_id)	none	Google / GA4 export	Immutable event_payload only via ; except the
stg_ga4__events	staging (view)	EVENT	surrogate event_id or natural (user_pseudo_id , event_timestamp , event_name)	â†’ src_ga4.events	analytics-eng / platform	1:1 typed, session_id, page_location, device_id, purchase_id, downstream
stg_ga4__event_params	staging (view)	EVENT — PARAM KEY	(event natural key, param_key)	â†’ src_ga4.events	analytics-eng / platform	Flattens the once so n
int_ga4__sessionized	intermediate (ephemeral/view)	SESSION	session_key	built from staging	analytics-eng	KEYSTON never ship (user_pseudo_id, landing_page, source/m fallback), engaged, ga_session_id
int_ga4__funnel_steps	intermediate (ephemeral/view)	SESSION	session_key	â†’ int_ga4__sessionized	analytics-eng	KEYSTON monotonic session_id
fct_sessions	mart / core (table)	SESSION (1 row/session)	session_key	user_pseudo_id â†’ dim_users.user_key ; date_key â†’ dim_date ; channel_key â†’ dim_channels	analytics-eng / Product Analytics	The core engagement
fct_funnel	mart / core (table)	SESSION (1 row/session)	session_key (also FK â†’ fct_sessions)	â†’ dim_users , dim_date , dim_channels	analytics-eng / Product Analytics	The PRIM queries (: wide dim: here.

Table	Layer	Grain	Primary Key	Foreign Keys	Owner / Steward	Why it exists
fct_daily_funnel	mart / growth (table)	DAY — [channel_group, device_category, country, is_new_user]	daily_funnel_key (md5 of grain)	date_key â†’ dim_date ; channel_key â†’ dim_channels	analytics-eng / Growth Analytics	Additive p revenue; 1 Decompo Rates NO layer).
fct_orders	mart / finance (table)	TRANSACTION (1 row/transaction_id)	order_key (= transaction_id)	session_key â†’ fct_sessions ; user_pseudo_id â†’ dim_users ; date_key â†’ dim_date ; channel_key â†’ dim_channels	analytics-eng / Finance & Revenue Analytics	The dedu entity-of-refund/sh revenue &
fct_order_items	mart / finance (table)	TRANSACTION — ITEM LINE (1 row/order-item)	order_item_key (md5 of transaction_id + item_id + row)	order_key â†’ fct_orders ; item_key â†’ dim_items	analytics-eng / Finance & Revenue Analytics	The explo revenue, i
fct_funnel_by_dim	mart / growth (table)	DAY — DIMENSION	composite (date_key , dimension , dimension_value)	date_key â†’ dim_date	analytics-eng / Growth Analytics	Funnel ro mix-vs-ra
fct_cohorts	mart / growth (table)	COHORT_WEEK — AGE_DAYS	composite (cohort_week , age_days [dims])	cohort_week derived from dim_users first-touch	analytics-eng / Growth Analytics	Weekly ac retained day_1/7, fct_cohc window ri cohorts.
dim_users	mart / core (table)	USER (1 row/user_pseudo_id)	user_key (= user_pseudo_id)	first_channel_key â†’ dim_channels	analytics-eng (conformed)	The USER touch attr is_purcha
dim_items	mart / core (table)	ITEM (1 row/item_id)	item_key (= item_id)	none	analytics-eng (conformed)	The confc (item_nan current pi
dim_channels	mart / core (table)	CHANNEL_GROUP (10 rows)	channel_key	none	analytics-eng (conformed)	Conforme default gr is_paid/is.
dim_date	mart / core (table)	DAY (1 row/day)	date_key	none	analytics-eng (conformed)	Conforme 31; week/mo for time-c

Grain discipline

Every table in the catalog declares **exactly one grain**, and that grain is its contract. The four irreducible grains are:

- **SESSION** — one analytical visit, `session_key`. Carried by `int_ga4_sessionized`, `int_ga4_funnel_steps`, `fct_sessions`, and `fct_funnel`. `fct_funnel` is the canonical session grain the semantic layer resolves `sessions`, `users`, conversion rates, and `revenue` against.
- **TRANSACTION** — one completed order, `order_key = transaction_id`. Carried by `fct_orders` and exploded one level down to TRANSACTION — ITEM LINE in `fct_order_items`.
- **ITEM LINE** — one product line within an order, `order_item_key`; the only grain at which product-level revenue and attach rate are valid.
- **COHORT — AGE** — one weekly acquisition cohort at a given `age_days`, `cohort_key`; the grain for retention curves.

Plus the conformed dimension grains (DAY, USER, CHANNEL_GROUP, ITEM) and the two pre-aggregated rollup grains (`fct_daily_funnel` at day — dims; `fct_funnel_by_dim` at day — one dimension).

Why mixing grains is forbidden. A metric is only additive and reconcilable at its declared grain. Summing `session_revenue` at order grain, or counting `orders` at session grain, double-counts or under-counts because the row multiplicity differs across facts. The semantic layer enforces this: each metric is bound to one grain (`revenue` $\hat{=}$ `fct_funnel` , `gross_revenue / aov` $\hat{=}$ `fct_orders` , `retention` $\hat{=}$ `fct_cohorts`), and cross-grain comparisons are only sanctioned at the *grand total* (e.g. session-grain `revenue` reconciles to order-grain `gross_revenue` to within 0.5% in aggregate, but can legitimately differ per segment because the joins differ). Declaring one grain per table $\hat{=}$ and never widening or mixing it $\hat{=}$ is what makes every governed metric re-aggregate correctly across any slice the Decompose agent requests, and is the precondition for the "0 hallucinated columns / 100% governed SQL" target.

Ownership model

analytics-eng owns the pipeline end-to-end $\hat{=}$ it builds and maintains every model from staging through the marts, owns all four conformed dimensions, and is accountable for the source-to-mart contracts. On top of that engineering ownership, each business domain assigns a **steward** who owns the *semantics* of a fact family: what the numbers mean, which caveats apply, and sign-off on changes that alter business meaning.

Steward	Stewards which tables	Domain accountability
Product Analytics	<code>fct_sessions</code> , <code>fct_funnel</code>	Session/funnel semantics: engagement definition, monotonic <code>reached_*</code> flags, conversion-rate definitions.
Finance & Revenue Analytics	<code>fct_orders</code> , <code>fct_order_items</code>	Revenue semantics: dedup-by-transaction, USD-only money, the gross/net/refund/shipping/tax split, AOV basis.
Growth Analytics	<code>fct_daily_funnel</code> , <code>fct_funnel_by_dim</code> , <code>fct_cohorts</code>	Aggregation and cohort semantics: additive-counts-only rule, mix-vs-rate decomposition inputs, retention denominators and the right-censoring caveat.
analytics-eng (conformed)	<code>dim_date</code> , <code>dim_users</code> , <code>dim_channels</code> , <code>dim_items</code>	Conformed dimension definitions: the 10 channel groups, the date spine, cookie-grain user identity, the product catalog.

What ownership means in practice. (1) *Schema changes* $\hat{=}$ adding, renaming, or retyping a column requires the owning steward's review, because downstream the semantic layer and the eval labels bind to physical names; a rename is a breaking change. (2) *Tests* $\hat{=}$ the owner is responsible for the dbt tests on their tables: grain-integrity (`unique` on the PK), `not_null` on keys, accepted-range on measures, funnel monotonicity, and revenue reconciliation. Keystone transforms (`int_ga4__sessionized` , `int_ga4__funnel_steps` , revenue dedup) get golden-value tests because they fail silently. (3) *SLAs* $\hat{=}$ owners commit to freshness (daily; available $\hat{=}$ 36h after `event_date` for the semantic-facing facts) and to the CI eval gate: no change to an owned model may drop top-1 diagnosis accuracy by more than the agreed threshold or introduce a hallucinated column. Conformed dimensions, owned solely by analytics-eng, carry the strictest change control because a change there ripples through every fact that joins them.

4. Event Model

The GA4 export is an **event stream**, not a table of business entities. Every row in `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` is **one event** $\hat{=}$ a single thing a user did (loaded a page, viewed a product, added to cart, paid). There is no session row, no order row, and no user row in the raw export; all of those are entities Helios *reconstructs* from the event stream. That is why the **event grain is the atomic foundation of the entire warehouse**: every fact and dimension downstream is a deterministic roll-up of these events, and if the event-access layer is wrong, every number above it is silently wrong. This section documents the event grain, its canonical taxonomy, the nested column shapes, the single flattening pattern Helios sanctions, and the two staging models (`stg_ga4__events` , `stg_ga4__event_params`) that are the only governed doorway between raw GA4 and everything else.

4.1 The event grain

property	value
Grain	one row per event (one user action at one microsecond timestamp)
Natural key	(<code>user_pseudo_id</code> , <code>event_timestamp</code> , <code>event_name</code> , <code>event_bundle_sequence_id</code>) $\hat{=}$ no enforced PK in raw
Partitioning	date-sharded tables <code>events_YYYYMMDD</code> (read via <code>_table_suffix</code>); spans <code>20201101</code> $\hat{=}$ <code>20210131</code>
Owner	Google / GA4 export (we own only the <i>access</i> layer, never the raw bytes)
Why it exists	the immutable source of truth ; every session, funnel flag, order, and item line is derived from it. We never query it directly except through staging.

A single visit ("session") is dozens of these event rows that happen to share a `ga_session_id` ; a single order is one (or, due to export retries, several) `purchase` rows that share a `transaction_id` . The event grain knows nothing about either $\hat{=}$ sessionization and order dedup are *our*

logic, applied in the intermediate and mart layers. The event row is deliberately "dumb and complete": it carries scalar context (`device.*` , `geo.*` , `traffic_source.*`), nested `event_params` , a nested `items` array, and a scalar `ecommerce` struct, so that any entity we need can be reconstructed without going back to a different source.

4.2 Canonical event taxonomy

The table below catalogs the canonical `event_name` values in the dataset: when each fires, the load-bearing `event_params` it carries, and the macro-funnel stage it maps to. The seven **bolded** stages plus `session_start` constitute the session-scoped macro funnel (`session_start` â†’ `view_item` â†’ `add_to_cart` â†’ `begin_checkout` â†’ `add_shipping_info` â†’ `add_payment_info` â†’ `purchase`); the remaining events are engagement/instrumentation signals that inform `engaged_session` and landing-page derivation but are not funnel steps.

event_name	fires when	key event_params	funnel stage / role
<code>session_start</code>	first event of a session	<code>ga_session_id</code> , <code>ga_session_number</code>	session_start (funnel denominator anchor)
<code>first_visit</code>	first-ever event for a <code>user_pseudo_id</code> cookie	<code>ga_session_id</code>	drives <code>is_new_user</code> / <code>ga_session_number = 1</code>
<code>page_view</code>	any page load	<code>page_location</code> , <code>page_title</code> , <code>page_referrer</code>	engagement; source of <code>landing_page</code>
<code>view_promotion</code>	promo/banner impression	<code>items[]</code> , <code>promotion_id</code>	top-of-funnel (pre- <code>view_item</code>)
<code>view_item_list</code>	category / search-results / list page view	<code>items[]</code> , <code>item_list_name</code>	top-of-funnel (pre- <code>view_item</code>)
<code>view_item</code>	product detail page (PDP) view	<code>items[]</code> , <code>page_location</code>	view_item
<code>select_item</code>	item clicked within a list	<code>items[]</code> , <code>item_list_name</code>	micro: list â†’ PDP
<code>add_to_cart</code>	item added to cart	<code>items[]</code> , <code>value</code> , <code>currency</code>	add_to_cart
<code>view_cart</code>	cart drawer / cart page viewed	<code>items[]</code> , <code>value</code>	micro: cart (between ATC and checkout)
<code>begin_checkout</code>	checkout initiated	<code>items[]</code> , <code>value</code> , <code>coupon</code>	begin_checkout
<code>add_shipping_info</code>	shipping tier entered	<code>shipping_tier</code> , <code>value</code>	add_shipping_info
<code>add_payment_info</code>	payment method entered	<code>payment_type</code> , <code>value</code>	add_payment_info
<code>purchase</code>	order completed	<code>transaction_id</code> , <code>value</code> , <code>items[]</code> , full <code>ecommerce</code> struct	purchase
<code>scroll</code>	90% scroll depth reached	<code>percent_scrolled</code>	engagement
<code>click</code>	outbound / UI click	<code>link_url</code> , <code>outbound</code>	engagement
<code>user_engagement</code>	foreground-engagement heartbeat	<code>engagement_time_msec</code> , <code>session_engaged</code>	drives <code>engaged_session</code> / <code>engagement_time_msec</code>

Two honesty notes that ride along with this taxonomy: (1) the funnel built from these events is **session-scoped** and uses **max-downstream** semantics (Section 5), so `view_cart` , `select_item` , and `view_promotion` are deliberately *not* macro-funnel steps â€” they are micro-signals the Critic can test but are excluded from the canonical six `reached_*` flags. (2) `session_engaged` and `engagement_time_msec` arrive on `user_engagement` , `page_view` , and other events, so engagement is a **session-level aggregate** of these params, not a property of any single event.

4.3 The nested event-row structure

A raw GA4 event row mixes four shapes. Understanding them is what makes the "flatten once" rule (Section 4.5) necessary rather than optional.

```
EVENT ROW (one user action)
â”€ scalar columns ..... user_pseudo_id, event_name, event_timestamp (micros),
â”€,
â”€ event_date, event_bundle_sequence_id, user_id (â”€ always NULL)
â”€ event_params ..... ARRAY<STRUCT<key STRING,
â”€,
â”€ value STRUCT<string_value, int_value, float_value, double_value>>>
â”€,
â”€ â”€ ga_session_id, ga_session_number, page_location, source, medium,
â”€,
â”€ campaign, engagement_time_msec, session_engaged, value, ...
â”€ items ..... ARRAY<STRUCT<item_id, item_name, item_brand, item_category,
â”€,
â”€ item_category2..5, item_variant, price_in_usd, price,
â”€,
â”€ quantity, item_revenue_in_usd, coupon>>
â”€,
â”€ â”€ present on view_item / add_to_cart / begin_checkout / purchase
```



```

user_pseudo_id,
{{ get_event_param('ga_session_id','int') }}          as ga_session_id,
{{ get_event_param('ga_session_number','int') }}      as ga_session_number,
event_name,
event_timestamp,
timestamp_micros(event_timestamp)                    as event_ts,
parse_date('%Y%m%d', event_date)                     as event_date,
{{ get_event_param('page_location') }}                as page_location,
{{ get_event_param('engagement_time_msec','int') }}   as engagement_time_msec,
{{ get_event_param('session_engaged') }}              as session_engaged,
-- session-scoped source/medium FIRST; user first-touch only as fallback (the gotcha, Å$5.3)
coalesce({{ get_event_param('source') }}), traffic_source.source) as source,
coalesce({{ get_event_param('medium') }}), traffic_source.medium) as medium,
{{ get_event_param('campaign') }}                    as campaign,
device.category                                       as device_category,
device.operating_system                             as operating_system,
device.web_info.browser                             as browser,
geo.country, geo.region,
ecommerce.transaction_id                             as transaction_id,
ecommerce.purchase_revenue_in_usd                   as purchase_revenue_in_usd
from {{ source('src_ga4','events') }}
where table_suffix between '20201101' and '20210131'

```

`stg_ga4__events` does **only** renames, casts, and the single param-extraction â€” no business logic, no sessionization, no aggregation. That discipline is what lets it stay a cheap `view` and what makes raw-schema changes a one-file fix.

`stg_ga4__event_params`

property	value
Layer	staging (materialized as <code>view</code>)
Grain	one row per event — param key
Primary key	(event natural key, <code>param_key</code>)
Foreign key	â†’ <code>src_ga4.events</code>
Owner / steward	analytics-eng / Platform
Why it exists	flattens the nested <code>event_params</code> ARRAY once into a long table so no downstream model ever re-UNNESTs ; the multi-key / exploratory complement to the scalar extractors on <code>stg_ga4__events</code> .

```

-- stg_ga4__event_params : 1 row per (event, param key), all typed slots exposed
select
to_hex(md5(user_pseudo_id || '-' || cast(event_timestamp as string) || '-' || event_name)) as event_key,
user_pseudo_id, event_name, event_timestamp,
ep.key          as param_key,
ep.value.string_value as string_value,
ep.value.int_value   as int_value,
ep.value.float_value  as float_value,
ep.value.double_value as double_value
from {{ source('src_ga4','events') }} , unnest(event_params) ep

```

Why flatten once, in staging

1. **Single typed-slot decision.** Choosing `int_value` vs `double_value` for a key is a correctness call; making it once (in the macro / staging) eliminates an entire class of silent type bugs.
2. **No fan-out drift.** The scalar-subquery pattern keeps `stg_ga4__events` 1:1 with raw; if every model re-unnested ad hoc, a stray `CROSS JOIN UNNEST` would multiply rows and quietly inflate counts.
3. **Cost.** The array scan is the expensive part of touching GA4. Doing it in one cheap `view` keeps the autonomous run inside its fixed byte budget rather than paying for the UNNEST in every mart.
4. **Schema-drift isolation.** When GA4 renames a struct or moves a param, exactly one staging model changes; the marts and the semantic layer are untouched.

This is the structural reason the event model is the atomic foundation: **everything above staging composes typed scalars, never raw arrays.**

5. Session Model

GA4 ships events, not sessions. The **session** is the first entity Helios reconstructs, and it is the spine of the whole product: the macro funnel, every rate metric, engagement, and most dimensional slicing all live at session grain. This section defines sessionization precisely, then documents the five session-grain models â€” `int_ga4__sessionized`, `fct_sessions`, `fct_funnel`, `fct_daily_funnel`, and `fct_funnel_by_dim` â€” each with grain, primary key, foreign keys, owner/steward, a "why it exists" line, and full column intent.

5.1 What a session is, and the `session_key`

A **session** = (`user_pseudo_id`, `ga_session_id`). `user_pseudo_id` is the device/cookie identifier; `ga_session_id` is GA4's per-cookie session counter, carried inside `event_params`. The canonical surrogate â€” used identically everywhere, never re-improvised â€” is:

```
session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
-- sessions = COUNT(DISTINCT session_key) -- never FARM_FINGERPRINT, never COUNT(*)
```

GA4 starts a new `ga_session_id` after 30 minutes of inactivity **and** at UTC midnight, so one human visit that straddles midnight splits into two sessions. A session is attributed to the **day of its first event** (`min(event_timestamp)`), so a midnight-crossing session counts once, on its start day. Rows with a NULL `ga_session_id` cannot be sessionized and are dropped at the intermediate layer.

Honesty note (cookie-grain identity). The user key is `user_pseudo_id`, a **device + browser cookie**, not a person. `user_id` is almost always NULL in this obfuscated export, so **no cross-device stitching** is possible: one human on phone + laptop is two users, and cookie churn (clearing cookies, Safari ITP / Firefox ETP ~7-day client-cookie caps, incognito) re-mints the same human as a new user. Every user-grain count inherits this bias; the Critic always caveats `users` / `new_users` / `returning_users` / ARPU as cookie approximations, never true person counts.

5.2 Derived session attributes

Sessionization reconstructs, per (`user_pseudo_id`, `ga_session_id`) group, the attributes GA4 never ships as a row:

- `landing_page` â€” the `page_location` of the **earliest-timestamp event** carrying one (typically `session_start` / `first_visit` / the first `page_view`).
- `session_start_micros` / `session_end_micros` â€” min / max of `event_timestamp`; `date_key` = `DATE(TIMESTAMP_MICROS(session_start_micros))`.
- `is_new_user` â€” `ga_session_number` = 1 (the cookie's first session); equivalently the session that contains `first_visit`.
- `ga_session_number` â€” the session ordinal for the cookie, bucketed downstream into `session_number_bucket`.
- `device_category` / `operating_system` / `browser` / `country` / `region` â€” any_value within the session (stable per visit).
- `engaged_session` â€” see Â§5.4.
- `channel_group` â€” derived from session `source` / `medium` via the single `channel_group_case()` macro (`macros/channel_group.sql`), the only place channel logic lives, producing exactly the 10 GA4 default groups.

5.3 Session-scoped source/medium and the `traffic_source` first-touch FALLBACK gotcha

This is the single most error-prone attribution detail in the dataset. The scalar `traffic_source` struct on every event is **USER FIRST-TOUCH** â€” the source/medium that *originally acquired the cookie*, repeated unchanged on every later event regardless of how the current session actually arrived. Using it as "the session's channel" mis-attributes every returning session to its original acquisition channel.

The governed rule: **prefer the session-scoped** `event_params.source` / `medium` (which reflect how *this* session arrived), and **fall back to the user-level** `traffic_source` struct only when the session params are NULL:

```
coalesce({{ get_event_param('source') }}, traffic_source.source, '(direct)') as source,
coalesce({{ get_event_param('medium') }}, traffic_source.medium, '(none)') as medium
```

This is the only sanctioned source/medium path. `channel_group` then flows from these session-scoped values through `channel_group_case()`. Carried caveat: because attribution is session-scoped occurrence, channel-sliced metrics answer "which channel was *this session* on", not "which channel deserves credit" â€” there is no last-non-direct or data-driven model, and (critically) no cost data, so ROI/ROAS is impossible from this layer.

5.4 Engaged session and new user

- `engaged_session` = `session_engaged` = '1' OR `engagement_time_msec` ≥ 10000 â€” GA4's own engaged-session bar (10+ seconds, or a conversion / multiple pageviews), aggregated to session grain (LOGICAL_OR of the flag; MAX of engagement time). It is a non-bounce / attention proxy, **not** funnel progress: an engaged session need not view a product, and `engagement_rate` = `engaged_sessions` / `sessions`.
- `is_new_user` = `ga_session_number` = 1. Honesty note: "new" is cookie-first-session, not first-time-human; cookie churn systematically inflates `new_users`.

5.5 int_ga4__sessionized â€” the sessionization keystone

property	value
Layer	intermediate (ephemeral / view ; not exposed to BI)
Grain	one row per session
Primary key	session_key
Foreign keys	built from stg_ga4__events (no mart FKs yet)
Owner / steward	analytics-eng
Why it exists	reconstructs the session row GA4 never ships by grouping events on (user_pseudo_id, ga_session_id); derives landing_page, session-scoped source / medium (with the traffic_source first-touch fallback), channel_group, device/geo, engaged_session, is_new_user, ga_session_number . KEYSTONE â€” if sessionization is wrong, every downstream number is silently wrong.

```
-- int_ga4__sessionized : 1 row per (user_pseudo_id, ga_session_id)
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string))) as session_key,
  user_pseudo_id, ga_session_id,
  any_value(ga_session_number) as ga_session_number,
  min(event_timestamp) as session_start_micros,
  max(event_timestamp) as session_end_micros,
  array_agg(page_location ignore nulls order by event_timestamp limit 1)[safe_offset(0)] as landing_page,
  -- session-scoped source/medium with user first-touch FALLBACK (the gotcha)
  coalesce(array_agg(source ignore nulls order by event_timestamp limit 1)[safe_offset(0)], '(direct)') as source,
  coalesce(array_agg(medium ignore nulls order by event_timestamp limit 1)[safe_offset(0)], '(none)') as medium,
  any_value(device_category) as device_category, any_value(operating_system) as operating_system,
  any_value(browser) as browser, any_value(country) as country, any_value(region) as region,
  countif(1) as event_count,
  max(coalesce(engagement_time_msec, 0)) as engagement_time_msec,
  logical_or(session_engaged = '1') as session_engaged_flag
from {{ ref('stg_ga4__events') }}
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```

5.6 The reached_* MAX-DOWNSTREAM monotonic flags

The funnel is computed in int_ga4__funnel_steps (session grain, PK session_key, FK â€” int_ga4__sessionized ; the second **KEYSTONE**) and carried onto fct_funnel. Each of the six reached_* flags uses **max-downstream** semantics: a session **reached stage X if it fired X or any later funnel-stage event** during the visit. Rolling each flag forward to every downstream stage makes the funnel **monotonic by construction**:

```
sessions â€” reached_view_item â€” reached_add_to_cart â€” reached_begin_checkout
â€” reached_add_shipping_info â€” reached_add_payment_info â€” reached_purchase
```

Because every downstream flag implies all upstream flags, a later-stage count can **never exceed** an earlier one, which is exactly what guarantees every step rate is â‰¥ 1 (e.g. view_to_cart_rate = add_to_cart_sessions / view_item_sessions â‰¥ 1). It also avoids dropping legitimate journeys that, say, re-add a cart item without re-firing view_item. (The retired did_* names are forbidden; an ordered/strict variant exists for the Critic to test step-skipping hypotheses but is not the default.) The canonical expression:

```
-- int_ga4__funnel_steps : 1 row per session, max-downstream (monotonic) flags
select
  to_hex(md5(user_pseudo_id || '-' || cast(ga_session_id as string))) as session_key,
  user_pseudo_id,
  true as did_session_start,
  logical_or(event_name in ('view_item', 'add_to_cart', 'begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase')) as reached_view_item,
  logical_or(event_name in ('add_to_cart', 'begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase')) as reached_add_to_cart,
  logical_or(event_name in ('begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase')) as
    reached_begin_checkout,
  logical_or(event_name in ('add_shipping_info', 'add_payment_info', 'purchase')) as
    reached_add_shipping_info,
  logical_or(event_name in ('add_payment_info', 'purchase')) as
    reached_add_payment_info,
  logical_or(event_name = 'purchase') as reached_purchase
from {{ ref('stg_ga4__events') }}
```

```
where ga_session_id is not null
group by user_pseudo_id, ga_session_id
```

5.7 fct_sessions â€” the conformed session entity-of-record

property	value
Grain	SESSION â€” one row per session
Primary key	session_key
Foreign keys	user_pseudo_id â†’ dim_users.user_key ; date_key â†’ dim_date.date_key ; channel_key â†’ dim_channels.channel_key
Owner / steward	analytics-eng / Product Analytics
Why it exists	the conformed SESSION entity-of-record : the wide session-dimension row (engagement + descriptive dims + funnel reach) that all session-grain analysis conforms to. Carries <code>engaged_sessions</code> .

column	type	key	intent
session_key	STRING	PK	TO_HEX(MD5(user_pseudo_id
user_pseudo_id	STRING	FK â†’ dim_users	device/cookie key (de-facto user)
ga_session_id	INT64		session id within the cookie
date_key	DATE	FK â†’ dim_date	session start date (from <code>min(event_timestamp)</code>)
session_start_ts / session_end_ts	TIMESTAMP		first / last event timestamp of the session
channel_key	STRING	FK â†’ dim_channels	session-scoped channel-group surrogate
source / medium / campaign	STRING		session-scoped acquisition (first-touch fallback)
landing_page	STRING		first <code>page_location</code> of the session
device_category / operating_system / browser	STRING		session device descriptors
country / region	STRING		session geo descriptors
is_new_user	BOOL		<code>ga_session_number = 1</code>
ga_session_number	INT64		session ordinal for the cookie
event_count	INT64		events in the session
engaged_session	BOOL		<code>session_engaged = '1'</code> OR <code>engagement_time_msec</code> $\geq$ 10000
engagement_time_msec	INT64		summed engagement time

5.8 fct_funnel â€” the semantic layer's primary session grain

property	value
Grain	SESSION â€” one row per session
Primary key	session_key (also FK â†’ fct_sessions)
Foreign keys	â†’ dim_users (user_pseudo_id), dim_date (date_key), dim_channels (channel_key)
Owner / steward	analytics-eng / Product Analytics
Why it exists	the PRIMARY session grain the semantic layer queries : the monotonic <code>reached_*</code> flags + deduped <code>session_revenue</code> + wide session dims, so <code>sessions</code> , <code>users</code> , the funnel rates, conversion, and <code>revenue_per_session</code> all resolve here. fct_funnel EXTENDS fct_sessions 1:1.

column	type	key	intent
session_key	STRING	PK / FK â†’ fct_sessions	session id (1:1 with fct_sessions)
user_pseudo_id	STRING	FK â†’ dim_users	user key
date_key	DATE	FK â†’ dim_date	session date
channel_key	STRING	FK â†’ dim_channels	channel group
device_category / country / is_new_user			wide dims, denormalized so the semantic layer slices without joins
did_session_start	BOOL		always TRUE â€” the denominator anchor (= sessions)
reached_view_item	BOOL		reached view_item or any later stage (max-downstream)
reached_add_to_cart	BOOL		reached add_to_cart or beyond
reached_begin_checkout	BOOL		reached begin_checkout or beyond
reached_add_shipping_info	BOOL		reached add_shipping_info or beyond
reached_add_payment_info	BOOL		reached add_payment_info or beyond
reached_purchase	BOOL		fired purchase
session_revenue	FLOAT64		SUM(purchase_revenue_in_usd) in the session, deduped to one value per transaction_id ; 0 for non-purchasing sessions

The 1:1 `fct_sessions` â†’ `fct_funnel` split is deliberate: `fct_sessions` is the descriptive session entity (engagement, dims), while `fct_funnel` carries the funnel + revenue payload at the same grain. The semantic layer points its `session` , `user` , and `revenue` (session-attributed) metrics at `fct_funnel` , computing every rate as $\text{SUM}(\text{numerator})/\text{SUM}(\text{denominator})$ over additive counts (e.g. $\text{session_conversion_rate} = \text{COUNTIF}(\text{reached_purchase}) / \text{COUNT}(\ast)$) so rates re-aggregate correctly across any dimension the Decompose agent slices.

5.9 `fct_daily_funnel` â€” pre-aggregated additive daily funnel

property	value
Grain	DAY Ã— [channel_group , device_category , country , is_new_user]
Primary key	daily_funnel_key (md5 of the grain columns)
Foreign keys	date_key â†’ dim_date ; channel_key â†’ dim_channels
Owner / steward	analytics-eng / Growth Analytics
Why it exists	additive pre-aggregated daily funnel counts + revenue ; the feed for Monitor (time-series anomaly), Decompose (mix-shift), and the eval injector. Aggregates <code>fct_funnel</code> (which carries <code>session_revenue</code>) â€” rates are NOT stored , they recompute in the semantic layer so they stay re-aggregatable across any slice.

column	type	intent
daily_funnel_key	STRING	md5 of grain columns (PK)
date_key	DATE	day
channel_key	STRING	channel group
device_category / country / is_new_user		grain dimensions
sessions	INT64	COUNT(DISTINCT session_key)
users / new_users / returning_users	INT64	distinct users (all / new / returning)
engaged_sessions	INT64	sessions where engaged_session
view_item_sessions	INT64	SUM(reached_view_item)

column	type	intent
add_to_cart_sessions	INT64	SUM(reached_add_to_cart)
begin_checkout_sessions	INT64	SUM(reached_begin_checkout)
add_shipping_info_sessions	INT64	SUM(reached_add_shipping_info)
add_payment_info_sessions	INT64	SUM(reached_add_payment_info)
purchasing_sessions	INT64	SUM(reached_purchase)
transactions	INT64	distinct transaction_id
revenue	FLOAT64	SUM(session_revenue)

Storing only additive counts (never rates) is the Simpson's-paradox defense: any rate the agents need is $\text{SUM}(\text{num})/\text{SUM}(\text{den})$ recomputed at the requested grain, so a daily-grain table rolls up to week/month or to any dimension subset without re-deriving rates from pre-divided ratios.

5.10 fct_funnel_by_dim “single-dimension funnel rollup

property	value
Grain	DAY — DIMENSION (one canonical dimension at a time)
Primary key	composite (date_key, dimension, dimension_value)
Foreign keys	date_key → dim_date
Owner / steward	analytics-eng / Growth Analytics
Why it exists	the funnel rolled up by a single canonical dimension (long/unpivoted: a dimension name + dimension_value); the direct input to the mix-vs-rate decomposition (decompose_change).

column	type	intent
date_key	DATE	day (PK part)
dimension	STRING	the dimension name (e.g. channel_group, device_category, country) → PK part
dimension_value	STRING	the value within that dimension → PK part
sessions	INT64	mix weight w_i for decompose_change
view_item_sessions → purchasing_sessions	INT64	the same additive reached_* counts as fct_daily_funnel
transactions	INT64	distinct transaction_id
revenue	FLOAT64	SUM(session_revenue)

Both fct_daily_funnel and fct_funnel_by_dim are **aggregations of fct_funnel**, **not FK children of it** → they materialize the same session-grain counts at coarser, agent-friendly grains. fct_funnel_by_dim deliberately holds one dimension at a time so the Decompose agent gets a clean (w_i, r_i) table per dimension: sessions is the mix weight, the reached_* counts give per-segment rates, and decompose_change splits any movement into mix / rate / interaction.

5.11 The session model as the analytical spine

Every model in this section traces back to one reconstruction → grouping events by (user_pseudo_id, ga_session_id) → and serves a distinct purpose: int_ga4__sessionized rebuilds the missing session row; int_ga4__funnel_steps adds the monotonic reach; fct_sessions is the conformed descriptive entity-of-record; fct_funnel is the 1:1 funnel + revenue extension that the **semantic layer queries directly**; and fct_daily_funnel / fct_funnel_by_dim pre-aggregate the same additive counts for Monitor, Decompose, and the eval injector. Honesty carries through the whole chain: identity is **cookie-grain** (no cross-device stitching), channel is **session-scoped occurrence with a first-touch fallback** (not attributed credit, and no cost data for ROI), and the ~3-month export window (Nov 2020 → Jan 2021) bounds every “returning”/cohort claim built on top of these sessions.

6. User Model

The user grain answers a different question than the session grain: not "what happened in this visit?" but "who is this person, where did they come from, and do they come back?" Helios resolves that grain with one conformed dimension (`dim_users`) and one acquisition-cohort fact (`fact_cohorts`). Both are built on a foundation that must be stated up front and carried into every user-level number: **identity in this dataset is cookie-grain, not person-grain.**

6.1 `dim_users` – the conformed USER entity-of-record

property	value
Grain	one row per <code>user_pseudo_id</code> (USER = device/cookie)
Primary key	<code>user_key</code> (= <code>user_pseudo_id</code>)
Foreign keys	<code>first_channel_key</code> -> <code>dim_channels.channel_key</code>
Owner / steward	analytics-eng (conformed dimension)
Materialization	<code>table</code>

Why it exists. GA4 ships events, not users. To answer "how many users did we acquire?", "what channel first brought them?", "is this a buyer?", and to provide a stable denominator for `revenue_per_user` (ARPU), Helios collapses the entire event stream to one row per `user_pseudo_id`. `dim_users` is the **single conformed user dimension** that `fact_sessions` and `fact_orders` join to (`user_pseudo_id` -> `user_key`) and that `fact_cohorts` rolls up from. Putting first-touch attribution and lifetime roll-ups here – rather than recomputing them per query – is what lets the semantic layer slice user metrics without re-scanning raw events.

column	type	key	description
<code>user_key</code>	STRING	PK	<code>user_pseudo_id</code> (the cookie/device id)
<code>first_touch_ts</code>	TIMESTAMP		<code>MIN(user_first_touch_timestamp)</code> – the user's first-ever event time
<code>first_touch_source</code>	STRING		session-scoped source of the user's first session (first-touch)
<code>first_touch_medium</code>	STRING		session-scoped medium of the first session
<code>first_channel_key</code>	STRING	FK -> <code>dim_channels</code>	<code>channel_group</code> surrogate of the first session (first-touch attribution)
<code>total_sessions</code>	INT64		<code>COUNT(DISTINCT session_key)</code> for the user
<code>total_revenue</code>	FLOAT64		<code>SUM(session_revenue)</code> (deduped) across the user's sessions
<code>is_purchaser</code>	BOOL		TRUE if <code>total_revenue > 0</code> / the user has ≥ 1 transaction
<code>is_new_user</code>	BOOL		TRUE if the user first appears in-window (first session has <code>ga_session_number = 1</code>)

```
-- dim_users : one conformed row per user_pseudo_id, first-touch attribution + lifetime roll-ups
with first_session as (
  -- the user's earliest session decides first-touch source/medium/channel
  select user_pseudo_id,
         array_agg(struct(source, medium, channel_group)
                   order by session_start_micros limit 1)[safe_offset(0)] as ft
  from {{ ref('fact_sessions') }}
  group by user_pseudo_id
)
select
  s.user_pseudo_id                                as user_key,
  timestamp_micros(s.first_touch_micros)          as first_touch_ts,
  any_value(fs.ft.source)                          as first_touch_source,
  any_value(fs.ft.medium)                         as first_touch_medium,
  any_value(c.channel_key)                        as first_channel_key,
  count(distinct s.session_key)                   as total_sessions,
  coalesce(sum(s.session_revenue), 0.0)           as total_revenue,
  coalesce(sum(s.session_revenue), 0.0) > 0       as is_purchaser,
  logical_or(s.ga_session_number = 1)             as is_new_user
from {{ ref('fact_sessions') }} s
join first_session fs using (user_pseudo_id)
left join {{ ref('dim_channels') }} c on c.channel_group = fs.ft.channel_group
group by 1
```

6.2 Cookie-grain identity – stated honestly

The user key is `user_pseudo_id`, the GA4 device/cookie identifier. The cross-device identifier `user_id` is **almost always NULL** in this obfuscated public dataset, so Helios performs **no identity stitching**. The consequences are not edge cases; they are structural, and the Critic agent attaches them to every user-grain finding:

- **A person on phone + laptop is two users.** Two cookies, two `user_pseudo_id`s, two rows in `dim_users`. There is no key that reunites them. `users`, `new_users`, and ARPU are therefore *device-cookie counts*, not headcounts, and are always over-stated relative to true humans.
- **Cookie churn inflates `new_users`.** Cleared cookies, Safari/ITP cookie expiry, incognito sessions, and re-consent all mint a fresh `user_pseudo_id`. A returning human on a new cookie looks simultaneously like a **churned user** (the old cookie went quiet) and a **brand-new acquisition** (the new cookie appears). New-vs-returning splits, and retention denominators, are biased by exactly this churn.
- **First-touch attribution lives on `dim_users`.** `first_touch_source` / `first_touch_medium` / `first_channel_key` capture the acquisition channel of the user's earliest session. This is deliberately a session-scoped attribution materialized once on the user row *not* the raw `traffic_source` struct. (The `traffic_source` gotcha: event-level `traffic_source.*` is itself GA4's first-touch attribution, so reading it per session mislabels every later session with the acquisition channel. `dim_users` is the *correct* home for first-touch; `fct_sessions` carries session-scoped source instead.)
- **New vs returning is `ga_session_number = 1`.** A session is a new-user session when its `ga_session_number` (the per-cookie session ordinal) equals 1 *equivalently*, when the `first_visit` event fired in it. `is_new_user` on `dim_users` is `LOGICAL_OR(ga_session_number = 1)` across the user's sessions; on `fct_sessions` / `fct_funnel` the same flag is per-session. Because the ordinal is per-*cookie*, churned-cookie returners are mis-classed as new *the same bias as above*, surfaced through a different column.

Honesty contract: any metric divided by `users` (ARPU, `new_users`, `returning_users`, all `day_N_retention`) is a **cookie-grain approximation** and must be narrated as such. Helios never claims person-level precision it does not have.

6.3 `fct_cohorts` “weekly acquisition cohorts feeding retention

property	value
Grain	one row per (<code>cohort_week</code> , <code>age_days</code>) [optionally x dims]
Primary key	composite (<code>cohort_week</code> , <code>age_days</code> [, dims])
Foreign keys	<code>cohort_week</code> derived from <code>dim_users.first_touch_ts</code> (cohort-membership rollup of <code>dim_users</code>)
Owner / steward	analytics-eng / Growth Analytics
Materialization	<code>table</code>

Why it exists. Retention is a *cohort* question “of the users we acquired in week W, how many came back by lifecycle-day N?” and that cannot be answered from `dim_users` (no time-since-acquisition axis) or `fct_sessions` (no fixed acquisition denominator). `fct_cohorts` is the purpose-built fact whose grain is (acquisition week x lifecycle age), pre-computing `cohort_size` and `retained_users_d1/d7/d30` so the semantic layer's `day_1_retention`, `day_7_retention`, and `day_30_retention` resolve as simple `SUM(retained_users_dN) / SUM(cohort_size)` aggregations. It is the grain named by those three metrics (`grain: fct_cohorts`) and the input to `stats-mcp.cohort_retention`.

column	type	key	description
<code>cohort_week</code>	DATE	PK	ISO-week (Monday) of <code>dim_users.first_touch_ts</code> “the acquisition cohort
<code>age_days</code>	INT64	PK	lifecycle day relative to first touch (0, 1, 7, 30 are the reported anchors)
<code>cohort_size</code>	INT64		distinct users first-touched in <code>cohort_week</code> “the FIXED denominator
<code>retained_users_d1</code>	INT64		users in the cohort with any session by lifecycle-day 1
<code>retained_users_d7</code>	INT64		users in the cohort with any session by lifecycle-day 7
<code>retained_users_d30</code>	INT64		users in the cohort with any session by lifecycle-day 30

How users roll in: every user lands in exactly one cohort, keyed by the ISO week (Monday) of `first_touch_ts` on `dim_users`. `cohort_size` is that week's distinct-user count and is **constant across all age rows of the cohort** (an invariant the semantic layer asserts). A user contributes to `retained_users_dN` if they have any session by lifecycle-day N “cumulative return by day N, not “active exactly on day N.” Because the denominator is the fixed original cohort (never the surviving population), retention is monotonically non-increasing by construction: `day_1_retention ≥ day_7_retention ≥ day_30_retention`.

```
-- fct_cohorts : cohort_week x age_days, fixed-denominator retention
with users as (
```

```

select user_key, date_trunc(date(first_touch_ts), week(monday)) as cohort_week,
       date(first_touch_ts) as ft_date
from {{ ref('dim_users') }}
),
returns as ( -- earliest session date per user, to test "returned by day N"
select user_pseudo_id as user_key,
       dt(date(timestamp_micros(session_start_micros))) over (partition by user_pseudo_id as ignore_me,
       date(timestamp_micros(session_start_micros)) as sess_date
from {{ ref('fct_sessions') }}
)
select
  u.cohort_week, ages.age_days,
  count(distinct u.user_key) as cohort_size,
  count(distinct if(date_diff(r.sess_date, u.ft_date, day) between 1 and 1, u.user_key, null)) as retained_users_d1,
  count(distinct if(date_diff(r.sess_date, u.ft_date, day) between 1 and 7, u.user_key, null)) as retained_users_d7,
  count(distinct if(date_diff(r.sess_date, u.ft_date, day) between 1 and 30, u.user_key, null)) as retained_users_d30
from users u
join returns r using (user_key)
cross join unnest([1,7,30]) as age_days with offset
group by u.cohort_week, ages.age_days

```

The ~3-month right-censoring caveat (carried, not hidden). The dataset spans only 2020-11-01..2021-01-31. `day_30_retention` is **observable only for cohorts acquired ≥ 30 days before the window's right edge**; cohorts acquired in the final weeks have not yet *had* 30 lifecycle days to return, so their d30 numerators are mechanically incomplete (right-censored), not low. Including immature cohorts in a $\text{SUM}(\text{retained_d30})/\text{SUM}(\text{cohort_size})$ over the whole window depresses the published rate artificially. Rules Helios enforces: `day_1_retention` is the least-censored (every cohort has a full 1-day horizon) and therefore the most trustworthy; `day_30_retention` must be restricted to *mature* cohorts; and small late cohorts are filtered by a minimum `cohort_size` threshold to keep tiny-cell noise out of the rate. The Critic flags any retention finding that mixes censored cohorts into a long-horizon rate.

7. Product Model

The product grain answers "which things did we sell, for how much, and how often do they appear in a basket?" Helios models it with the conformed `dim_items` catalog, an SCD2 snapshot (`snap_dim_items`) that gives that catalog history, and the `fct_order_items` line-item fact that explodes the purchase `items[]` array.

7.1 `dim_items` – the conformed PRODUCT catalog

property	value
Grain	one row per <code>item_id</code> (ITEM = sellable product)
Primary key	<code>item_key</code> (= <code>item_id</code>)
Foreign keys	none (a leaf conformed dimension)
Owner / steward	analytics-eng (conformed dimension)
Materialization	<code>table</code>

Why it exists. Item descriptors (`item_name` , `item_brand` , the `item_category` hierarchy) are repeated on *every* `items[]` element of *every* purchase/view event. Storing them once in a conformed catalog dimension – and joining `fct_order_items` to it on `item_key` – keeps the line-item fact narrow, gives every product a single canonical name/brand/category, and lets the semantic layer slice product metrics by `item_category` / `item_brand` without re-parsing arrays. It is the conformed product dimension shared across all product analysis.

column	type	key	description
<code>item_key</code>	STRING	PK	<code>item_id</code>
<code>item_name</code>	STRING		<code>items.item_name</code> (latest known)
<code>item_brand</code>	STRING		<code>items.item_brand</code>
<code>item_category</code>	STRING		primary category (<code>items.item_category</code>)
<code>item_category2..5</code>	STRING		the GA4 category hierarchy levels 2-5
<code>current_price_in_usd</code>	FLOAT64		latest observed <code>items.price_in_usd</code>

7.2 snap_dim_items â€” SCD2 history of price & category

`dim_items` holds the *current* state. But a product's price and category are **slowly-changing attributes**: a SKU can be re-priced, re-merchandised, or moved between categories within the analysis window. If a Diagnose finding says "AOV fell because Category X products dropped in price on Dec 12," the *current* price in `dim_items` cannot support that claim â€” the historical price is gone. `snap_dim_items` is the dbt **SCD2 snapshot** that preserves this history.

property	value
Grain	one row per (<code>item_id</code> , validity interval)
Primary key	(<code>item_key</code> , <code>dbt_valid_from</code>)
Tracked columns	<code>price_in_usd</code> , <code>item_category</code> (+ hierarchy), <code>item_name</code>
Owner / steward	analytics-eng
Materialization	dbt <code>snapshot</code> (check strategy)

```
-- snapshots/snap_dim_items.sql : SCD2 on price/category drift
{% snapshot snap_dim_items %}
{{ config(target_schema='helios_snapshots', unique_key='item_key',
        strategy='check',
        check_cols=[ 'price_in_usd', 'item_category', 'item_category2', 'item_name' ]) }}
select item_id as item_key, item_name, item_brand,
       item_category, item_category2, price_in_usd
from {{ ref('stg_ga4_purchase_items') }}
{% endsnapshot %}
-- dbt adds dbt_valid_from / dbt_valid_to / dbt_scd_id; a new row is cut whenever a tracked col changes.
```

Why it exists. It is the only place the model can answer time-travel product questions â€” point-in-time price, "what category was this SKU in on date D," whether an AOV/mix movement was a *price* change versus a *quantity/mix* change. `dim_items` (current state) is the default join for descriptive slicing; `snap_dim_items` (historical state) is consulted when a finding's correctness depends on the value *as of* a past date.

7.3 fct_order_items â€” the exploded line-item fact

property	value
Grain	one row per (<code>transaction_id</code> , item line)
Primary key	<code>order_item_key</code> = MD5(<code>transaction_id</code> + <code>item_id</code> + line row)
Foreign keys	<code>order_key</code> -> <code>fct_orders.order_key</code> ; <code>item_key</code> -> <code>dim_items.item_key</code>
Owner / steward	analytics-eng / Finance & Revenue Analytics
Materialization	<code>table</code>

Why it exists. Order-level totals on `fct_orders` cannot tell you *which products* drove revenue or how baskets are composed. `fct_order_items` explodes the purchase `items[]` array into one row per line, giving the product-level grain that `product_revenue` , `items_per_transaction` , and `product_attach_rate` resolve against. It is the bridge between the transaction fact (`order_key`) and the product catalog (`item_key`).

column	type	key	description
<code>order_item_key</code>	STRING	PK	md5(<code>transaction_id</code> + <code>item_id</code> + <code>row_number</code>) surrogate
<code>order_key</code>	STRING	FK -> <code>fct_orders</code>	the transaction this line belongs to
<code>item_key</code>	STRING	FK -> <code>dim_items</code>	the product on this line
<code>item_name</code>	STRING		denormalized for convenience
<code>item_category</code>	STRING		denormalized for convenience
<code>quantity</code>	INT64		units on this line (<code>items.quantity</code>)
<code>item_revenue_in_usd</code>	FLOAT64		line merchandise revenue (<code>items.item_revenue_in_usd</code>)
<code>price_in_usd</code>	FLOAT64		unit price (<code>items.price_in_usd</code>)
<code>coupon</code>	STRING		line-level coupon if present

7.4 How product metrics derive from the line grain

All product metrics are additive sums or order-distinct counts over `fct_order_items`:

- `product_revenue` = `SUM(item_revenue_in_usd)` (group by `item_key / item_category`). Merchandise value only â€” **excludes shipping/tax, is not refund-adjusted** â€” so it reconciles to `gross_revenue` at the grand total but is *not* identical line-by-line.
- `items_per_transaction` = `SUM(total_item_quantity) / COUNT(DISTINCT order_key)` (quantity per order).
- `product_attach_rate` = `COUNT(DISTINCT order_key with the item) / COUNT(DISTINCT order_key over ALL orders)` â€” basket penetration. **Order-distinct, not quantity-weighted** (three units count once), and the denominator is *all* orders, so attach rates sum past 100% across products. Mixing it onto a non-line grain double-counts multi-line orders â€” the reason it is pinned to `fct_order_items`.

7.5 The item-level view/purchase caveat

A product is commonly **viewed in one session and bought in a later one** (often on a different cookie/device â€” see Â§6.2). Therefore product-level *view* metrics and product-level *purchase* metrics live at different grains and must not be silently divided into a single-session "product conversion":

- The **purchase** side (`fct_order_items`) is anchored on `transaction_id` and the buying session/order. It is where `product_revenue`, units, and attach rate are exact.
- The **view** side comes from the `view_item items[]` array at the session grain, where the same product may appear across many sessions and users.

Because the view and the purchase need not share a session or even a cookie, Helios computes product *revenue/units/attach* from the item-line grain and treats any product-level *view-to-purchase* rate as a cross-session approximation that the Critic caveats â€” never a within-session conversion. This mirrors the cookie-grain honesty of Â§6: the demand signal is real, but the linkage between a specific view and a specific purchase is not guaranteed.

8. Order Model

8.1 fct_orders â€” the deduped order header

property	value
Grain	one row per <code>transaction_id</code> (TRANSACTION)
Primary key	<code>order_key</code> (= <code>transaction_id</code>)
Foreign keys	<code>session_key</code> -> <code>fct_sessions</code> ; <code>user_pseudo_id</code> -> <code>dim_users</code> ; <code>date_key</code> -> <code>dim_date</code> ; <code>channel_key</code> -> <code>dim_channels</code>
Owner / steward	analytics-eng / Finance & Revenue Analytics
Materialization	<code>table</code>

Why it exists. GA4's `purchase` events are *not* clean order rows: the same `transaction_id` can appear on multiple event rows, and order-level totals are buried in the nested `ecommerce` struct. `fct_orders` is the **deduped order header â€” the TRANSACTION entity-of-record** â€” exactly one row per `transaction_id`, with merchandise/refund/shipping/tax totals lifted out of `ecommerce`. It is the grain the semantic layer's `transactions`, `gross_revenue`, `net_revenue`, and `aov` resolve against, and the conformed transaction node every finance metric joins from.

column	type	key	description
<code>order_key</code>	STRING	PK	<code>transaction_id</code>
<code>session_key</code>	STRING	FK -> <code>fct_sessions</code>	originating session
<code>user_pseudo_id</code>	STRING	FK -> <code>dim_users</code>	buyer (cookie-grain)
<code>date_key</code>	DATE	FK -> <code>dim_date</code>	purchase date
<code>channel_key</code>	STRING	FK -> <code>dim_channels</code>	attributed channel
<code>order_ts</code>	TIMESTAMP		purchase event timestamp
<code>gross_revenue</code>	FLOAT64		<code>ecommerce.purchase_revenue_in_usd</code> (merchandise; excludes shipping/tax)
<code>refund_value_in_usd</code>	FLOAT64		refund on the order (usually 0/NULL)

column	type	key	description
net_revenue	FLOAT64		gross_revenue - COALESCE(refund_value_in_usd, 0)
shipping_value_in_usd	FLOAT64		stored separately; never folded into revenue
tax_value_in_usd	FLOAT64		stored separately; never folded into revenue
total_item_quantity	INT64		units in the order
unique_items	INT64		distinct items in the order

8.2 transaction_id dedup and NULL handling

GA4 exports emit **duplicate purchase rows** for one `transaction_id` (client retries, multi-stream collection, replays). A naive `SUM(purchase_revenue_in_usd)` double-counts. The fix: collapse to one row per `transaction_id` first (`ANY_VALUE` of each order-level field, which is identical across the dupes), *then* aggregate. **NULL transaction_id** rows are purchases GA4 failed to tag (test orders, mis-instrumented streams); they are excluded from `transactions` and `gross_revenue` but logged by the Critic as a data-quality caveat â€” a spike in NULL-id purchases can masquerade as a revenue drop.

```
-- fct_orders : one deduped row per transaction_id; net = gross - refund
with purch as (
  select
    ecommerce.transaction_id          as order_key,
    {{ session_key('user_pseudo_id', 'ga_session_id') }} as session_key,
    user_pseudo_id,
    timestamp_micros(event_timestamp) as order_ts,
    any_value(ecommerce.purchase_revenue_in_usd) as gross_revenue, -- identical across dupe rows
    any_value(ecommerce.refund_value_in_usd)    as refund_value_in_usd,
    any_value(ecommerce.shipping_value_in_usd)  as shipping_value_in_usd,
    any_value(ecommerce.tax_value_in_usd)       as tax_value_in_usd,
    any_value(ecommerce.total_item_quantity)    as total_item_quantity,
    any_value(ecommerce.unique_items)          as unique_items
  from {{ source('src_ga4', 'events') }}
  where event_name = 'purchase'
    and ecommerce.transaction_id is not null -- drop untagged purchases
  group by order_key, session_key, user_pseudo_id, order_ts -- collapse duplicate purchase rows
)
select
  order_key, session_key, user_pseudo_id, date(order_ts) as date_key, order_ts,
  gross_revenue,
  coalesce(refund_value_in_usd, 0.0) as refund_value_in_usd,
  gross_revenue - coalesce(refund_value_in_usd, 0.0) as net_revenue,
  coalesce(shipping_value_in_usd, 0.0) as shipping_value_in_usd,
  coalesce(tax_value_in_usd, 0.0) as tax_value_in_usd,
  total_item_quantity, unique_items
from purch
```

8.3 gross vs net vs shipping/tax

- **gross_revenue** = deduped `SUM(purchase_revenue_in_usd)` â€” merchandise value, the headline `revenue`.
- **net_revenue** = `gross_revenue - refund_value_in_usd`. Refunds are near-zero in the sample window, so `gross_revenue ~= net_revenue`, but the distinction is preserved for the dollar-at-risk quantification every finding carries.
- **shipping/tax excluded.** `shipping_value_in_usd` and `tax_value_in_usd` are stored on `fct_orders` but **never folded into revenue or aov**, so AOV reflects merchandise value, not the total a customer paid. All money uses `_in_usd` columns only; the non-USD twins (`purchase_revenue`, `price`) are never aggregated.

8.4 revenue (session-attributed) vs gross_revenue (order grain) â€” reconciliation

Helios carries the same dollars at two grains, and they must reconcile at the grand total:

- **revenue / session_revenue** lives on `fct_funnel` (session grain): the deduped purchase revenue *attributed to the session in which it occurred*. This is what `revenue_per_session` and the funnel's revenue-at-risk use, because it sits beside the funnel flags.
- **gross_revenue** lives on `fct_orders` (transaction grain): the deduped order header total.

Both dedup by `transaction_id` and use `purchase_revenue_in_usd`, so `SUM(session_revenue)` over `fct_funnel` equals `SUM(gross_revenue)` over `fct_orders` at the grand total (within the 0.5% `reconcile` tolerance). They differ only in *grain and attribution axis* â€” session-scoped funnel analysis vs order-scoped finance analysis â€” never in the underlying dollars. A drift between them signals a dedup or join bug and fails the finding.

8.5 Relationships

```
erDiagram
    fct_sessions ||--o{ fct_orders : "session_key (0..N orders, usually 0 or 1)"
    fct_orders ||--}| fct_order_items : "order_key (1..* line items)"
    fct_orders }o--|| dim_users : "user_pseudo_id"
    fct_orders }o--|| dim_date : "date_key"
    fct_orders }o--|| dim_channels : "channel_key"
    fct_order_items }o--|| dim_items : "item_key"
```

- **fct_sessions 1 --- 0..N fct_orders** (via `session_key`): a session yields zero or more orders â€” usually 0 (most sessions never purchase) or 1, occasionally more. This is why session-to-order joins are LEFT joins and `session_revenue` defaults to `0.0`.
- **fct_orders 1 --- * fct_order_items** (via `order_key`): every order explodes into one or more line items, the bridge into `dim_items` and the basis for all product-level revenue.

Together these close the chain: `dim_users` / `dim_date` / `dim_channels` (conformed) -> `fct_sessions` -> `fct_orders` -> `fct_order_items` -> `dim_items`, with the same deduped dollars reconciling from the session funnel down to the product line.